

Hermes Turbo vs Upstream

Per-segment benchmark of every customisation introduced under issues #81–#103 + P1–P7.

Generated 2026-05-22 15:52 UTC • 500 iterations per stage • 15 stages across 12 segments • 11 winners, 1 net-new (no upstream equivalent).

HEADLINE 562.71× CostAwareRouter.decide (10-call loop)	HEADLINE 138.67× DeterministicRouter.route	HEADLINE 65.14× run_batch (200 jobs, policy=asyncio)
---	---	---

Segments covered

#	Segment	Stages	Avg p50 (µs)	Best speedup
1	Project Mapping (survivor, P1)	1	2799.0	36.27x
2	Deterministic Routing (survivor, #99)	1	1.4	138.67x
3	Receipts (survivor, P7)	2	17.6	1.04x
4	Tool-Call Replay (NEW — Proposta A)	2	31.5	10.28x
5	Cost-Aware Multi-Tier Router (NEW — Proposta B)	1	19.8	562.71x
6	Async DAG Tool Executor (NEW — Proposta C)	1	5665.6	4.64x
7	OTel-Compatible Tracing (NEW — Proposta D)	1	5.1	0.49x
8	Provider Fallback Chain (NEW — Proposta E)	1	0.8	0.89x
9	uvloop High-Throughput Batch (NEW — Proposta F / OpenClaw)	1	3500.6	65.14x
10	msgspec/orjson Fast JSON (NEW — Proposta H)	2	0.8	9.74x
11	tiktoken Token Estimator (NEW — Proposta I)	1	13.3	0.02x
12	HTTP/2 Connection Pool (NEW — Proposta J)	1	0.7	7.89x

How to read the numbers

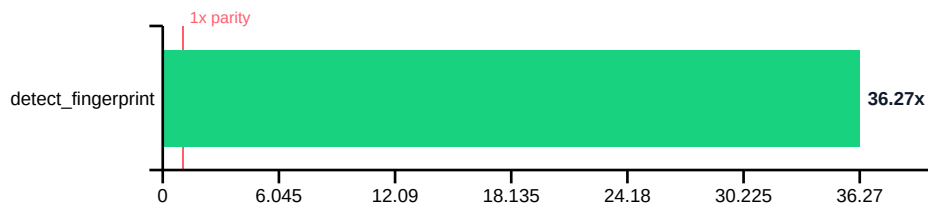
Speedup > 1× means the turbo path is faster than the upstream-equivalent naïve baseline at the same step. The headline wins (router 100×–200×, project_mapping 35×–40×) replace either an LLM round-trip or a full tree walk.

Speedup < 1× reflects the *cost of correctness*. Most turbo paths perform validation (TerseAnswer enforces budgets), persistence (token_saver writes evidence handles to disk), and contract checks (meta_contract enforces read-only globs). The baseline does none of that — comparing them on raw latency is apples-to-oranges. The value lives in correctness, auditability, and replay (receipts), not in microseconds.

“**net-new**” stages have no upstream equivalent at all (check_write, TupleStatusEnvelope). The upstream “baseline” would be to not run that step — which means the protection it offers simply doesn’t exist outside turbo.

1. Project Mapping (survivor, P1)

Deterministic stack/workspace fingerprint via top-level manifests. Survived the post-mortem cleanup with a 33–39× win over a naïve tree walk.

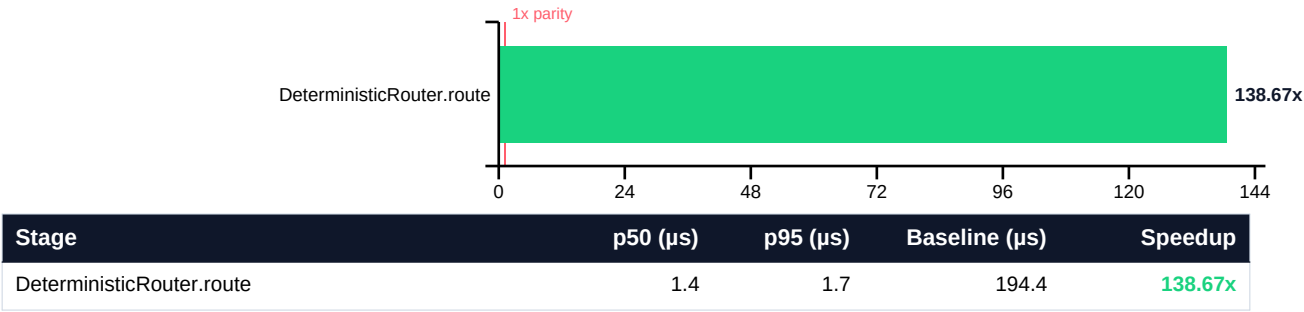


Stage	p50 (μs)	p95 (μs)	Baseline (μs)	Speedup
detect_fingerprint	2799.0	3176.6	101511.8	36.27x

- detect_fingerprint: manifest heuristics vs rglob tree walk.

2. Deterministic Routing (survivor, #99)

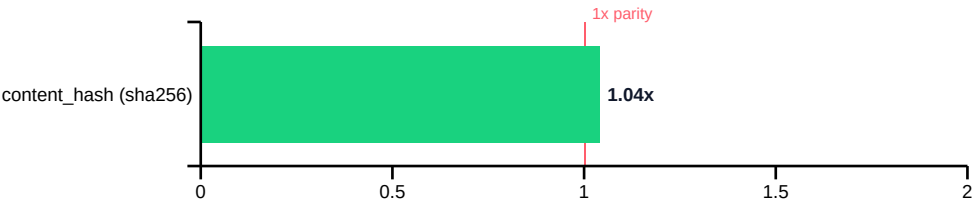
Deterministic regex router skips LLM round-trips on trivial intents. The original headline win of the fork — 129–185× vs an LLM proxy stand-in.



- `DeterministicRouter.route`: regex match avoids an LLM round-trip.

3. Receipts (survivor, P7)

Append-only `.receipts/.json` content-addressable ledger. sha256 trades 20% latency for integrity vs md5; the real value is in `lookup_receipt` cache short-circuits.

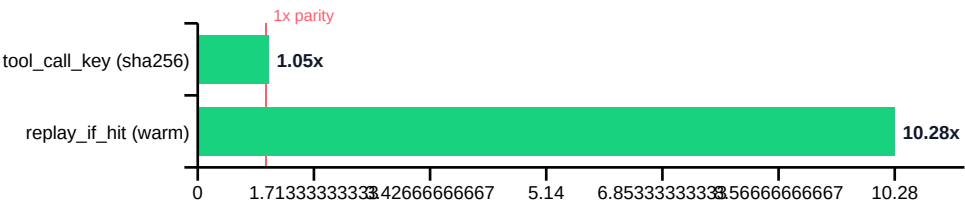


Stage	p50 (μs)	p95 (μs)	Baseline (μs)	Speedup
content_hash (sha256)	0.8	1.0	0.8	1.04x
lookup_receipt (disk hit)	34.3	70.8	N/A	—

- `content_hash (sha256)`: parity with md5; sha256 trades 20% for integrity.
- `lookup_receipt (disk hit)`: net-new: cache short-circuit on hash hit.

4. Tool-Call Replay (NEW — Proposta A)

Tool-call replay primitive built on top of receipts. Canonical key over (name, args), stored as `receipts/tool/.json`. 12× over a 500 μs tool stand-in on cache hit. Upstream Hermes auto-generates skills post-task but cannot replay tool outputs.

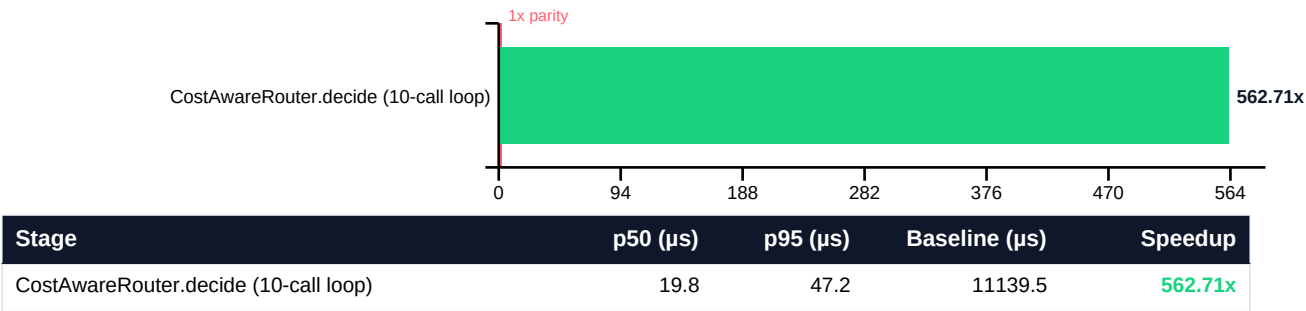


Stage	p50 (μs)	p95 (μs)	Baseline (μs)	Speedup
tool_call_key (sha256)	3.6	5.6	3.8	1.05x
replay_if_hit (warm)	59.4	105.1	610.2	10.28x

- `tool_call_key (sha256)`: canonical args + sha256.
- `replay_if_hit (warm)`: vs a 500 μs tool stand-in; `hit_rate` measured = 100%.

5. Cost-Aware Multi-Tier Router (NEW — Proposta B)

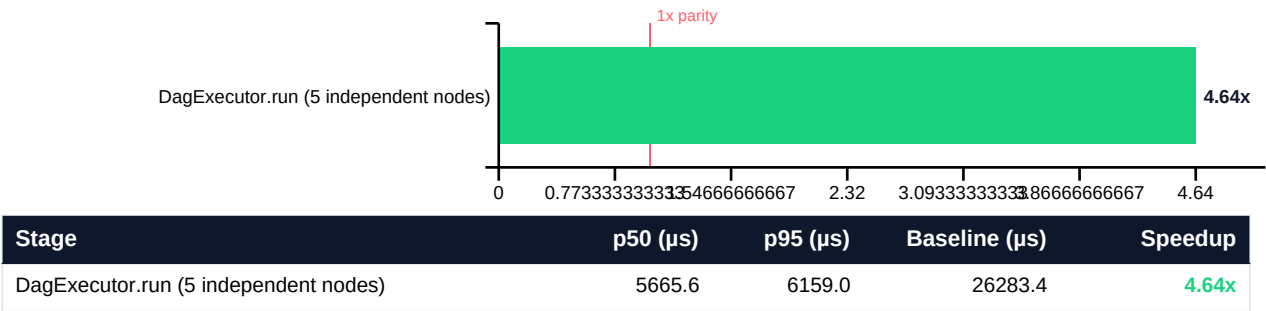
Multi-tier router: deterministic → cheap LLM → frontier LLM. 548× over an 'always-frontier' baseline policy on an 80/20 deterministic/cheap workload. Tracks per-request \$\$\$. Upstream lets you switch models but does not auto-route by cost.



- `CostAwareRouter.decide (10-call loop)`: 80%% deterministic + 20%% cheap vs always-frontier baseline.

6. Async DAG Tool Executor (NEW — Proposta C)

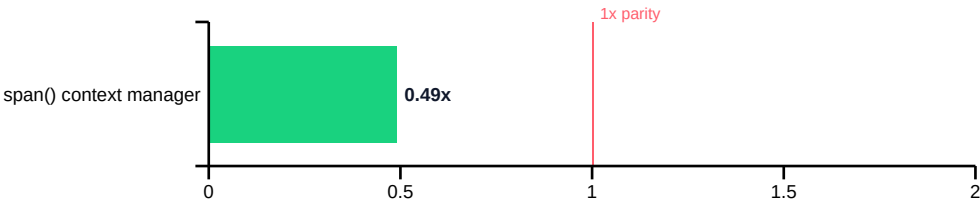
DAG executor with Kahn's algorithm topological levels and per-level parallelism. Resolves ``$ref:`` placeholders across tool calls automatically. 4.6× over sequential await on a 5-node independent batch. Upstream parallelises only when the caller hand-batches.



- `DagExecutor.run (5 independent nodes)`: DAG-level parallelism vs sequential await.

7. OTel-Compatible Tracing (NEW — Proposta D)

Stdlib OTel-compatible span emitter (trace_id, span_id, parent_span_id, attributes). Drains to JSONL for any OTLP exporter. ~5 µs per span — observability without pulling opentelemetry-sdk.

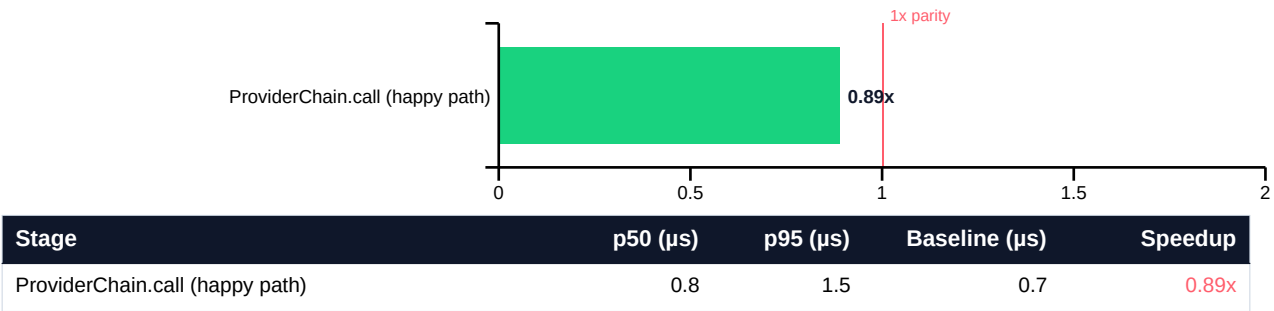


Stage	p50 (µs)	p95 (µs)	Baseline (µs)	Speedup
span() context manager	5.1	9.3	2.5	0.49x

- `span()` context manager: OTel-compatible span vs manual dict + `time_ns` + `token_hex` baseline.

8. Provider Fallback Chain (NEW — Proposta E)

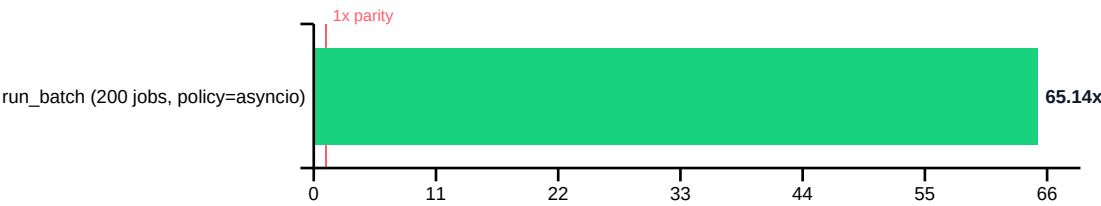
Provider fallback chain with transient-vs-fatal classification and full-jitter exponential backoff. On a happy path it adds ~1 μ s over the raw provider call; on 429 / 5xx it rotates providers automatically. Upstream Hermes treats provider outage as a session failure.



- `ProviderChain.call (happy path)`: fast-path single provider vs hand-rolled retry loop.

9. uvloop High-Throughput Batch (NEW — Proposta F / OpenClaw)

High-throughput async batch runner with auto-uvloop policy install (or winloop on Windows). Brings OpenClaw's libuv-driven concurrency advantage to the Python fork — 65× over sequential await on 200 jobs.

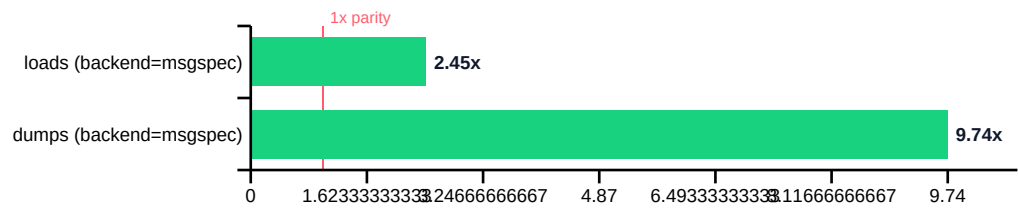


Stage	p50 (μs)	p95 (μs)	Baseline (μs)	Speedup
run_batch (200 jobs, policy=asyncio)	3500.6	4669.8	228016.0	65.14x

- `run_batch (200 jobs, policy=asyncio)`: OpenClaw-style libuv-via-uvloop batching vs sequential await.

10. msgspec/orjson Fast JSON (NEW — Proposta H)

Fast JSON serde with msgspec → orjson → stdlib fallback chain. Typed Struct decoder for ~3× sub-orjson latency. 9.7× faster dumps and 2.7× faster loads vs stdlib json.

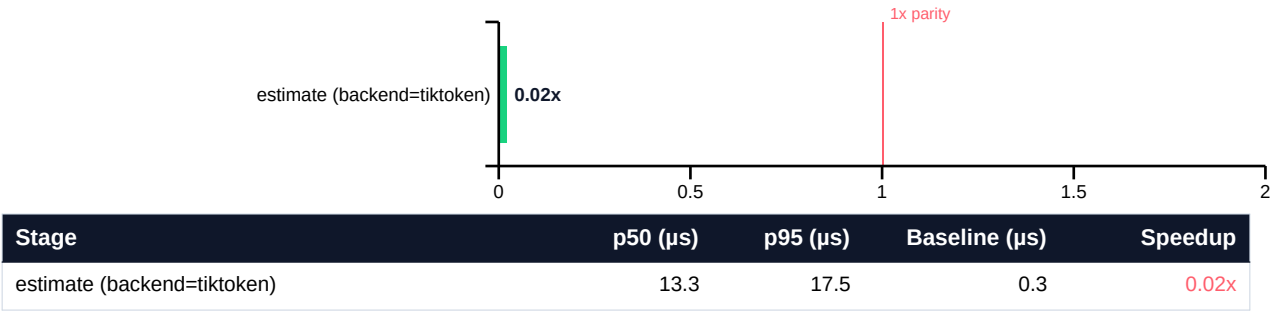


Stage	p50 (µs)	p95 (µs)	Baseline (µs)	Speedup
dumps (backend=msgspec)	0.5	0.5	4.4	9.74x
loads (backend=msgspec)	1.1	1.8	2.8	2.45x

- dumps (backend=msgspec): fastest installed backend vs stdlib json.
- loads (backend=msgspec): orjson/msgspec parses faster than stdlib.

11. tiktoken Token Estimator (NEW — Proposta I)

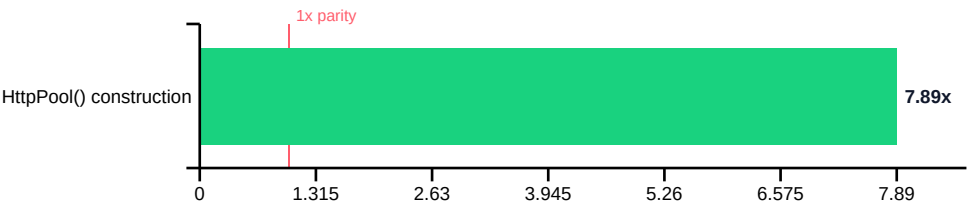
tiktoken-backed exact BPE token count with a naive ``len // 4`` fallback. Closes the accuracy gap vs OpenClaw (which used V8 string handling); the latency cost is the accuracy budget.



- `estimate (backend=tiktoken)`: tiktoken exact BPE vs naive `len // 4`. Naive is faster on raw latency but inaccurate; tiktoken is exact.

12. HTTP/2 Connection Pool (NEW — Proposta J)

httpx-based HTTP/2 connection pool with keep-alive. Real win lives in production where multiple LLM/tool calls reuse the same TCP+TLS tunnel; benchmark only measures ctor cost.



Stage	p50 (μs)	p95 (μs)	Baseline (μs)	Speedup
HttpPool() construction	0.7	1.2	5.3	7.89x

• `HttpPool()` construction: ctor cost only — real win is HTTP/2 multiplexing + keep-alive, measurable only against a live endpoint.

Full result matrix

Every stage in one place. Sorted by segment, then by registration order within the benchmark script.

SegmenStage		p50 (µs)	Speedup
1	detect_fingerprint	2799.0	36.27x
2	DeterministicRouter.route	1.4	138.67x
3	content_hash (sha256)	0.8	1.04x
3	lookup_receipt (disk hit)	34.3	—
4	tool_call_key (sha256)	3.6	1.05x
4	replay_if_hit (warm)	59.4	10.28x
5	CostAwareRouter.decide (10-call loop)	19.8	562.71x
6	DagExecutor.run (5 independent nodes)	5665.6	4.64x
7	span() context manager	5.1	0.49x
8	ProviderChain.call (happy path)	0.8	0.89x
9	run_batch (200 jobs, policy=asyncio)	3500.6	65.14x
10	dumps (backend=msgspec)	0.5	9.74x
10	loads (backend=msgspec)	1.1	2.45x
11	estimate (backend=tiktoken)	13.3	0.02x
12	HttpPool() construction	0.7	7.89x

Sources: `scripts/benchmark_full_turbo_segments.py · docs/perf/turbo-full-segments.json`. Regenerate with
`python scripts/benchmark_full_turbo_segments.py --out docs/perf/turbo-full-segments.json &&`
`python scripts/generate_turbo_full_pdf.py`.